Ecole Supérieure Privée Technologie & Ingénierie



A.U.2022/2023

Programmation Python

ING TIC-1 / Cours intégré (31h50 Cours/TP)

Dr. Ing. Sawssen JALEL
sawssen.jalel@tek-up.tn

Chapitre 1:
Les notions de base de Python

Plan



1. Python : Introduction
2. Installation & IDEs
3. Syntaxe et types de base
4. Les entrées/sorties
5. Les structures de contrôle & itératives
6. Les fonctions: Définition & Appel
7. Structure générale d’un programme

Python: Introduction

Un peu d'historique

Guido van Rossum



- Le langage de programmation Python a été créé en **1989** par Guido van Rossum, aux Pays-Bas.
- La première version publique de ce langage a été publiée en **1991.**
- La dernière version de Python est la version 3.11.1

Release:	3.11.1
Date:	January 12, 2023
Editor:	Pablo Galindo Salgado

- La **Python Software Foundation** est l'association qui organise le développement de Python et anime la communauté de développeurs et d'utilisateurs.

Caractéristiques de Python

- **Multiplateforme :** C'est-à-dire qu'il fonctionne sur de nombreux systèmes d'exploitation : Windows, Mac OS X, Linux, Android, iOS.
- **Gratuit.**
- C'est un langage **de haut niveau**.
- **Langage interprété :** Un script Python n'a pas besoin d'être compilé pour être exécuté, contrairement à des langages comme le C ou le C++.
- Programmation **orientée objet**, **procédurale** et **fonctionnelle**.
- Un langage à **typage dynamique** (pas de déclaration de types).
- N'utilise **aucun pointeur**.

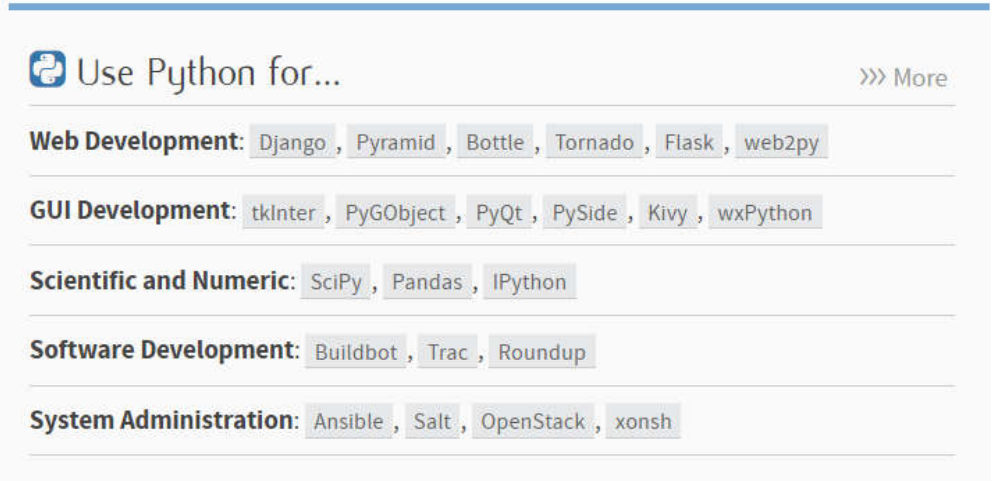
Caractéristiques de Python

- **Complet** : sa bibliothèque standard permet de manipuler des expressions régulières, de manipuler des fichiers, d'utiliser différents protocoles Internet, etc.
- Développé par une communauté **extrêmement dynamique.**
- Gestion automatique de la mémoire (comptage de références)
- Très bien **documenté**.



Avantages de Python

- **Qualité :** produire facilement du code évolutif et maintenable et offre les avantages de la programmation orientée-objet.
- **Productivité** Python permet de produire rapidement du code compréhensible.
- **Portabilité** La disponibilité de l'interpréteur sur de nombreuses plates-formes.
- **Intégration** L'utilisation de Python est parfaitement adaptée à l'intégration de composants écrits dans un autre langage de programmation (C, C++, Java…).



Bonnes pratiques en Python

Format du code



- **Un code python suit une certaine syntaxe et une mise en page.**
- **L'indentation** a une importance capitale dans ce langage et elle doit être de 4 espaces.
- Une ligne doit contenir 80 caractères maximum.
- Ajoutez deux lignes vides entre deux éléments de haut niveau (des classes par exemples), pour plus d'ergonomie.
- Séparez chaque fonction par une ligne vide.
- Les noms (variables, fonctions, classes,…) ne doivent pas contenir d'accent. Que des lettres et des chiffres.

Bonnes pratiques en Python

Nommage

- **Noms des modules et packages**: Les modules et packages doivent avoir des noms courts, constitués de lettres minuscules. Les noms de modules peuvent contenir des signes_(souligné).

- **Noms de classes:** Les noms de classes sont écrits en minuscules, exceptée la première lettre de chaque mot qui la constitue. Exemple: *MaClasse*.

- **Noms de variables, fonctions et méthodes**: Le nom est entièrement écrit en minuscules et les mots sont séparés par des signes soulignés (_). Exemple :*nom_de_fonction*.

- **Constantes**: Les constantes doivent être écrites entièrement en majuscules, les mots étant séparés par un signe souligné (_). Exemple :*NOM_DE_MA_CONSTANTE*.

Bonnes pratiques en Python

Commentaires

- Les commentaires en Python commencent avec un caractère dièse, **#**, et s'étendent jusqu'à la fin de la ligne. Un commentaire peut apparaître au début d'une ligne ou à la suite d'un espace ou de code, mais pas à l'intérieur d'une chaîne de caractères littérale.

```
# Ceci est un commentaire
```

- Une variante consiste à commencer un commentaire avec trois guillemets **"""** et à le finir de la même façon. C'est une façon aisée d'écrire un long commentaire qui court sur plusieurs lignes.

```
" " " ceci est un commentaire
                    sur deux lignes " " "
```

Particularités de Python

- **Toute** variable est un **objet**, même les types **de base.**

Un objet B possède:

- **identité id** (adresse mémoire), **id(B).**
- **Un type** : (intrinsèque : int, float, str, bool... Ou définit par l'utilisateur à l'aide d'une classe) ,**type(B).**
- Contient des données (**valeur**).
- L'opérateur **is** compare l'identité de 2 objets (adresses).
- L'opérateur **==** compare le contenu de deux objets.

Particularités de Python

- L'**indentation** est cruciale;
- La syntaxe **minimise** l'emploi de **caractères** superflus :
  - Pas de **points-virgules**;
  - Pas d'**accolades** pour les blocs;
  - Pas de **parenthèses** superflues, etc.
- On **ne déclare jamais** le type d'une variable, celui-ci est **inféré** (**typage dynamique**);
- En revanche, le typage est **fort** (opérations **interdites** entre types non compatibles).
- Python est doté d'une **gestion automatique de la mémoire** par ramasse-miettes;
- On utilise l'extension **.py** pour les fichiers Python.

Mots réservés

and as assert break	class continue def del	elif else except	False Finally for from global	if import in is lambda	None nonlocal not or
pass raise return	True try	while with	yield		

```
>>> global = 2300
SyntaxError: invalid syntax
```

Modules & Scripts

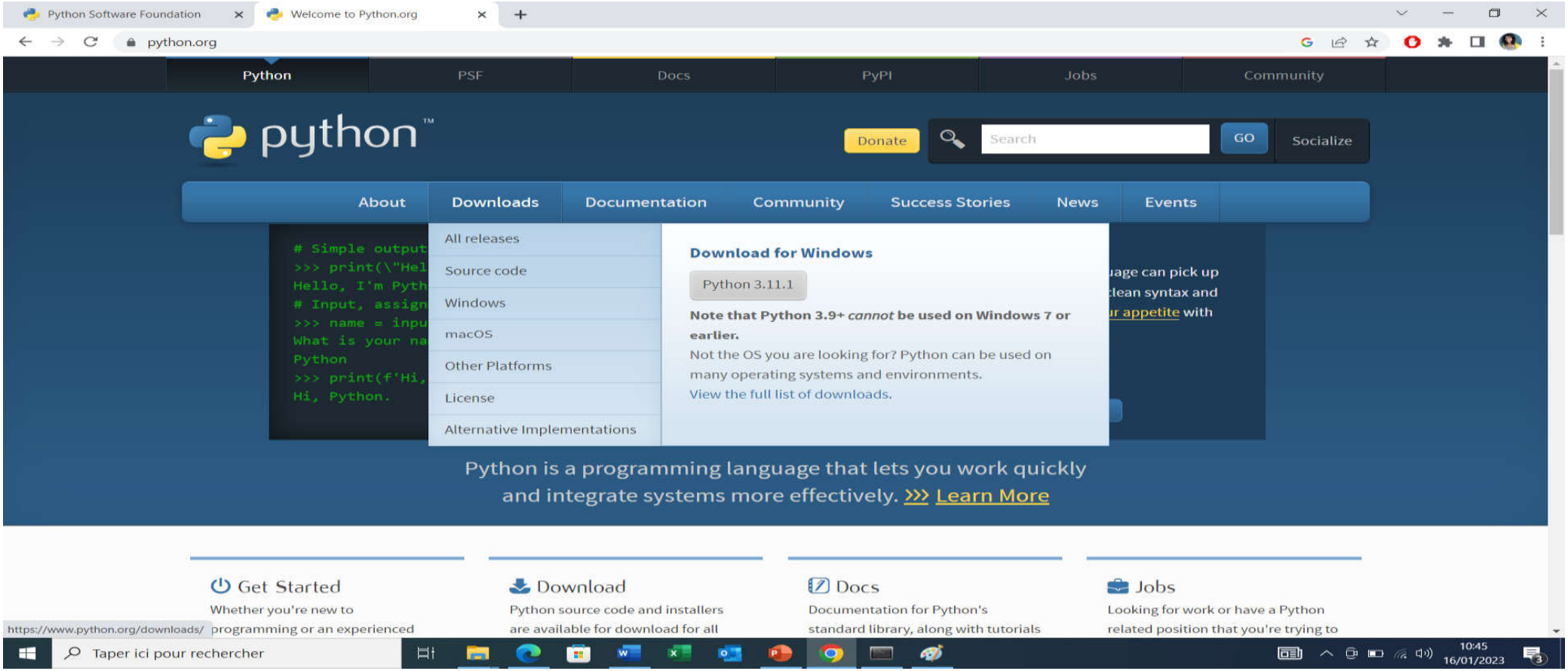
- Les **programmes (scripts)** et les **modules** python sont différenciés seulement par la manière d'appel :

  - Les fichiers *.py qui sont précédés par **import** sont des **Modules.**

  - Tous les autres fichiers *.py sont des **programmes** qui **s'exécutent directement.**

Installation & IDEs

python.org



Le site de Python met à notre disposition les distributions officielles.

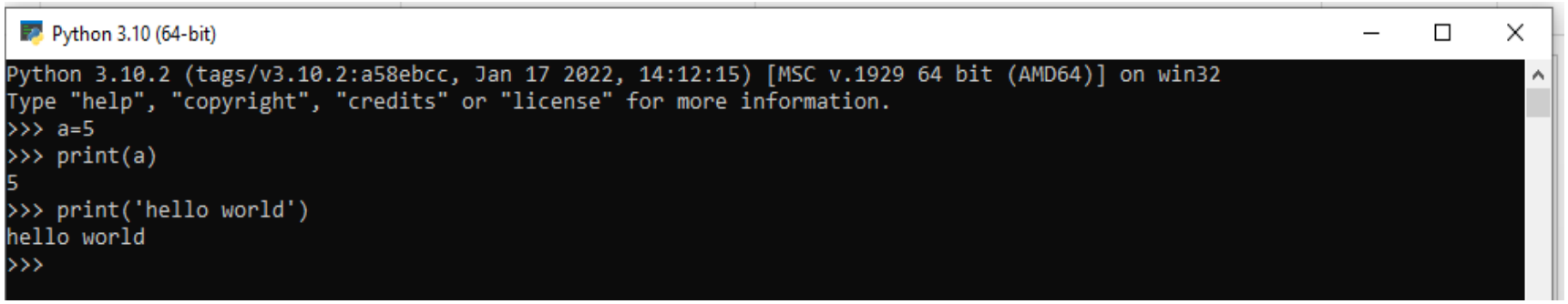
Utilisation de Python

- Il existe deux modes de programmation avec python :

1. **Mode interactif**
2. script / programme

- **Mode interactif :**
  - Il y a un dialogue entre l'utilisateur et l'interprète : les commandes entrées par l'utilisateur sont évaluées au fur et à mesure.

  - Cette solution est pratique pour **réaliser des prototypes**, ainsi que pour **tester** tout ou partie d'un programme ou plus simplement pour **interagir** aisément et rapidement avec des **structures de données complexes**.



Utilisation de Python

- Il existe deux modes de programmation avec python :

1. Mode interactif
2. **script / programme**

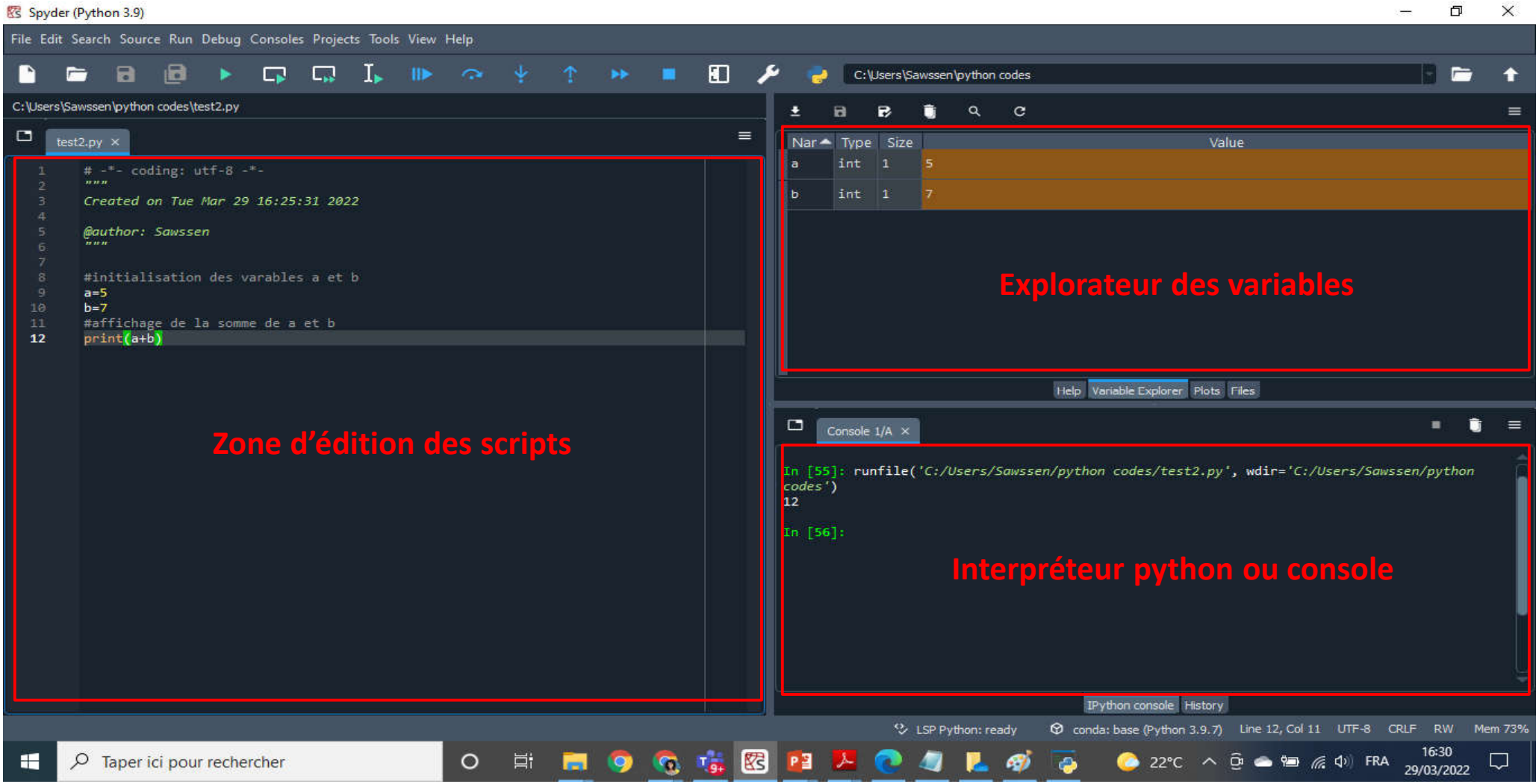
- **Mode script / programme :**

  - Pour une utilisation en mode script, les instructions à évaluer par l'interprète sont sauvegardées, dans un fichier dont l'extension est .py

  - On saisit l'intégralité des instructions, puis demander leur exécution à l'interprète.

```
 8   #initialisation des varables a et b
 9   a=5
10   b=7
11   #affichage de la somme de a et b
12   print(a+b)
```

Environnement de travail Spyder



Syntaxe et types de base

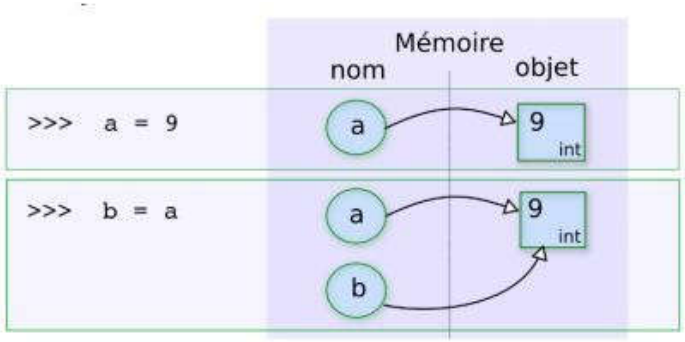
Python: Tout est objet

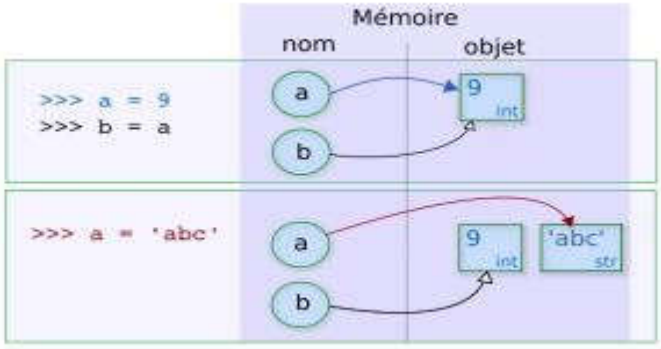


Un objet :

- possède une **identité** (adresse mémoire) ;
- possède un **type** : un objet est **l'instanciation d'une classe** qui définit son type (type interne : int, float, str… ou type utilisateur : classxxx) ;
- contient des données (exple : objet numérique –> sa valeur).

Affectation référence -> objet





L'affectation opère de droite à gauche :

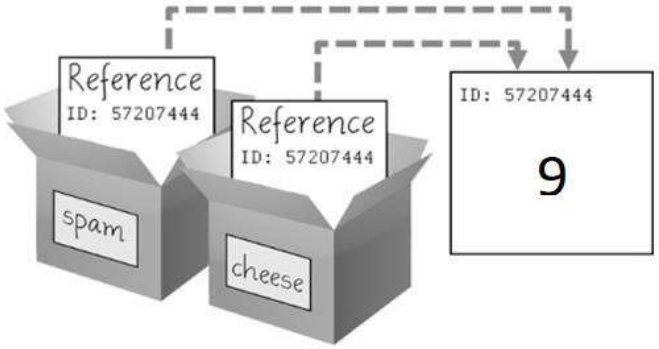
- le terme de droite est une **expression évaluée en tant qu'objet**
- le terme de gauche est l'**identificateur** (référence, nom, étiquette) affecté à l'objet évalué
- C'est l'objet qui porte le type et les données (la valeur, pour un objet numérique).

- Un objet **ne peut pas changer d'identité** (adresse mémoire), ni de type et il **peut avoir plusieurs noms** (alias).

- Un identificateur associé d'abord à un objet peut ensuite référencer un nouvel objet !

- Quand un objet n'a plus de nom (nombre de références nul), il est détruit (mécanisme automatique de **"ramasse-miettes"**, garbage collector).

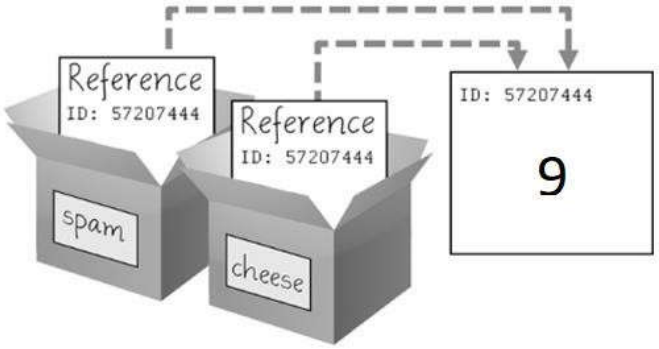
affectation référence -> objet



- Un objet **ne peut pas changer d'identité** (adresse mémoire), ni de type.

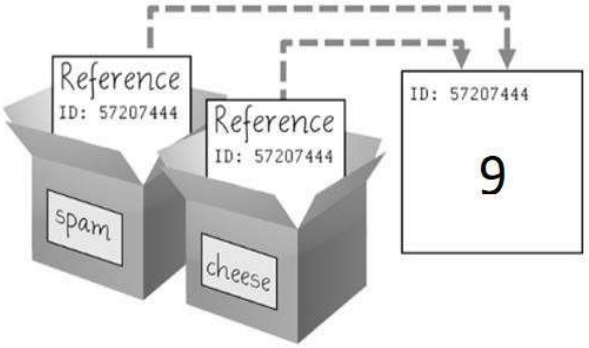
- Un objet **peut avoir plusieurs noms** (alias).

Affectation référence -> objet



- L'affectation **=** permet d'affecter un nom à un objet créé ou existant.
- **type (objet)** renvoie le type de l'objet. Le type d'un objet détermine : ses valeurs possibles, ses opérations supportées, la signification des opérations supportées (opérations polymorphes).
- **id (objet)** renvoie l'identité de l'objet. Un identificateur associé d'abord à un objet peut ensuite référencer un nouvel objet !

Affectation référence -> objet



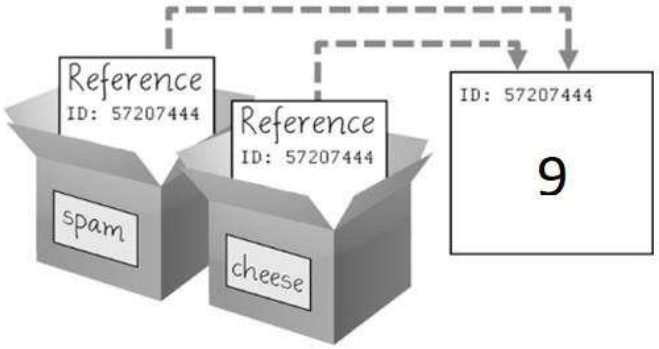
- L'opérateur **==** teste l'égalité des valeurs de 2 objets.

```
>>> variable_1 = 500
>>> variable_2 = 500
>>> variable_1 == variable_2
True
```

- L'opérateur **is** compare l'identité de 2 objets (Le `is` est un opérateur d'identité utilisé pour vérifier si deux variables pointent vers le même objet en mémoire ou pour comparer l'identité de deux objets. Il renvoie `True` si les opérandes sont identiques (se réfèrent au même objet).

```
>>> variable_1 = 500
>>> variable_2 = 500
>>> variable_1 is variable_2
False
```

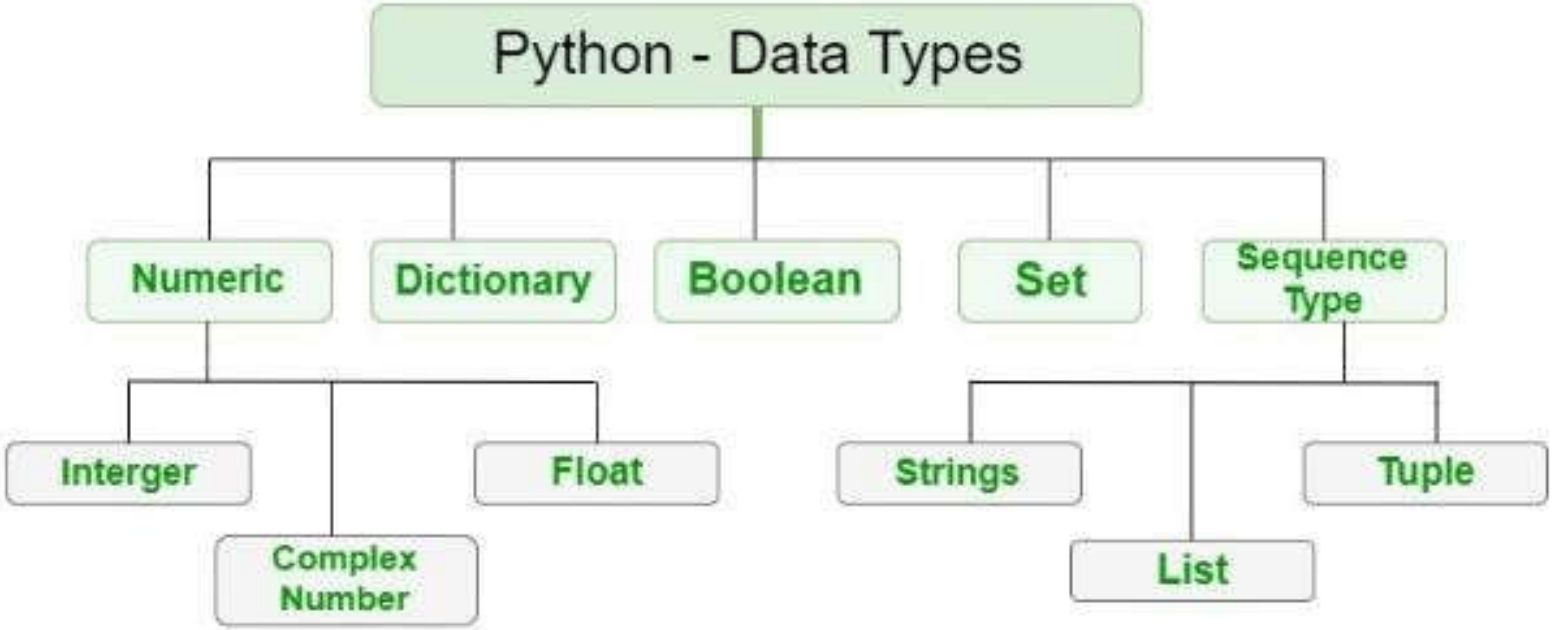
Affectation multiple




```
# assigner une valeur à plusieurs variables
>>> x = y = 7
>>> x
7
>>> y
7


#  effectuer des affectations parallèles
>>> a, b = 4, 8.33
>>> a
4
>>> b
8.33
```

Types de données



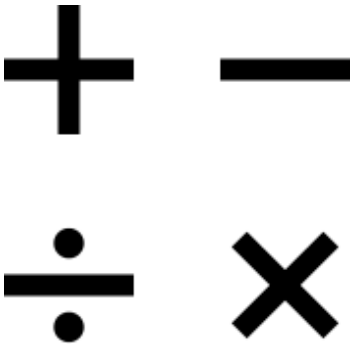
Les types élémentaires

1. **Les types numériques**
   - Le type **int** (x=156)
   - Le type **float** (x=1.49)
   - Le type **complex** (z=8+9j) : z.real; z.imag; z.conjugate()

2. **Le NONE TYPE:** La valeur nulle (None)

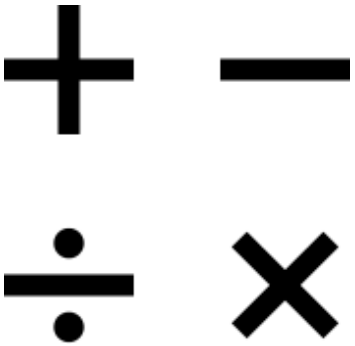
1. **Le type BOOLEAN**: Deux valeurs possibles True et False (1/0)

Opérations mathématiques



- opérations de base comme en C : `x+y`, `x-y`, `x * y`...
- `x ** y` ou `pow(x,y)` : x^y
- `x/y` : Division de x par y
- `x//y` : Division **entière** de x par y
- `int(x)` : cast en entier
- `float(x)` : cast en float
- `complex(re, im)` : faire un nombre complexe
- plus d'opérations : `import math as m`

Opérations mathématiques



Priority	Operator	
1	`**`	
2	`+`, `-` (note: unary operators located next to the right of the power operator bind more strongly)	unary
3	`*`, `/`, `//`, `%`	
4	`+`, `-`	binary

Opérateurs booléens

```
and
or
not
```

Comparaisons :

```
<
<=
>
>=
==
!=
```

Test sur les objets :

```
is
is not
```

Conversion de type (cast)

Fonction	Rôle	Exemple
int(x)	Convertir x à un entier	x = 1.2 int(x) x = Int("12")
float(x)	convertir x à un nombre réel	float("3.14")
str(15.5)	Convertir vers une chaine de caractères	str(15.5)

Les entrées/sorties

Saisie du clavier: La fonction input()

- La fonction standard **input()** effectue **toujours** une saisie en mode texte (la saisie est une chaîne de catactères).

- Utilisation du **transtypage (conversion de type)** pour entrer des int, float …

```
In [80]: a = input("Entrez une donnée numérique : ")

Entrez une donnée numérique : 10

In [81]: type(a)
Out[81]: str

In [82]: b=float(a)

In [83]: type(b)
Out[83]: float

In [84]: c = float(input("Entrez une donnée numérique : "))

Entrez une donnée numérique : 7

In [85]: type(c)
Out[85]: float
```

Affichage au clavier: La fonction print()

• La fonction **print()** est une fonction de sortie : elle affiche à l'écran une donnée ou le contenu d'une variable.

```
8   a, b = 2, 5
9   print(a,b)
10  print('Somme: ', a+b)
11  print('Le produit de ', a, ' par ' , b, ' vaut : ', a*b)
12  print() # Affiche une nouvelle ligne
13  # Pour afficher un espace à la place d'une nouvelle ligne
14  print(a, end=" ")   # 2 (et ne va pas à la ligne)
15  print(a,b, sep="*") # 2*5
16
```

```
2 5
Somme:  7
Le produit de  2  par  5  vaut :  10

2 2*5

In [124]:
```

Les structures de contrôles & répétitives

Les instructions conditionnelles

Syntaxe

```
if condition1:
        instruction 1
elif condition2:
        instruction 2
elif condition3:
        instruction 3
        instruction 4
else :
        instruction 5
        instruction 6
```

Bloc d'instructions à exécuter si la condition 1 est vraie.

Bloc d'instructions à exécuter si la condition 2 est vraie.

Bloc d'instructions à exécuter si la condition 3 est vraie.

Bloc d'instructions à exécuter si aucune des conditions précédentes n'est vraie.

PS: Attention à l'indentation

Exemple

```
x=1
#Condition1:
if x>0 :
  print(x,"est positif")
#Condition 2:
elif x<0 :
  print(x,"est négatif")
#Condition 3:
else:
  print(x,"est nul")
```